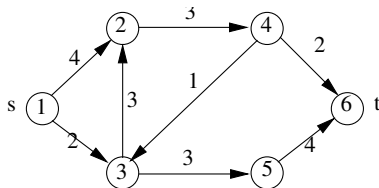


Maximális folyam 1

Szerző: Horváth Gyula
Előadó: Varga Péter

2026. május 10.



1. ábra

Bemenet: $G = (V, E, c)$ irányított, súlyozott gráf, az s forrás és t nyelő (cél) pont. A $c : E \rightarrow \mathbf{N}$ kapacitás függvény minden élhez pozitív egész értéket rendel.

Kiszámítandó olyan $f : E \rightarrow \mathbf{N}$ függvény, a **folyam**, amelyre teljesül az alábbi két feltétel:

Kapacitás korlát $(\forall e \in E) (0 \leq f(e) \leq c(e))$

Megmaradás

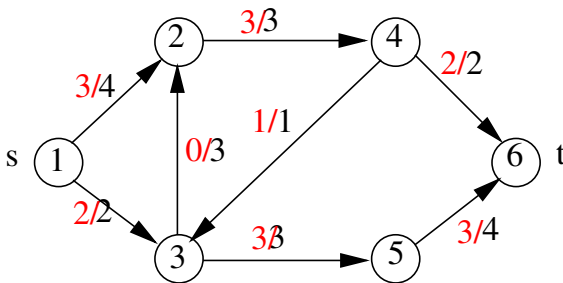
$$(\forall v \in V) (v \neq s, v \neq t) (\sum_u f(u, v) = \sum_u f(v, u))$$

Ha a két feltétel teljesül, akkor azt mondjuk, hogy a folyam **megengedett**.

A teljes **folyam** értéke:

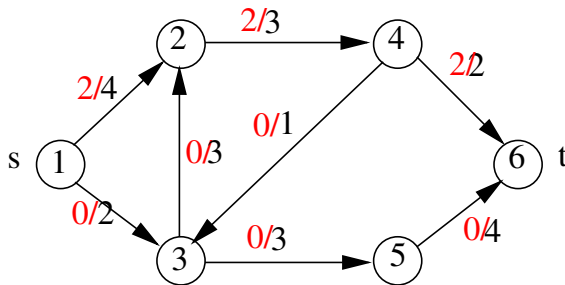
$$|f| = \sum_{u \in V} f(s, u) - \sum_{u \in V} f(u, s)$$

Az a cél, hogy olyan folyamot adjunk, amelyre az $|f|$ teljes folyam értéke maximális.



2. ábra. Egy maximális folyam: $|f| = 5$

Legyen $s \rightsquigarrow t$ egy tetszőleges út, és k az útban lévő élek kapacitásának minimuma. Ekkor az a folyam, amelyre teljesül, hogy $f(u, v) = k$ ha $u \rightarrow v$ él az út egy éle és $f(u, v) = 0$ ha az él nincs az útban, megengedett folyam lesz.



3. ábra. Egy megengedett folyam

Legyen f egy tetszőleges megengedett folyam, $\pi = s \rightsquigarrow t$ egy út és $k > 0$.

Ha π út minden $u \rightarrow v$ éle esetén a folyam értékéhez hozzáadunk k -t úgy, hogy teljesüljön a kapacitás korlát, tehát $f(u, v) + k \leq c(u, v)$ teljesüljön, akkor ismét megengedett folyamat kapunk. Ekkor azt mondjuk, hogy a π út növelő út k növekménnyel.

Nyilvánvaló hogy elvégezve a növelést a π úton, a megmaradás feltétel továbbra is teljesül, tehát megengedett folyam lesz.

Ezen észrevételből következik, hogy az f folyam nem lehet maximális, ha van növelő út.

Látható, hogy a $s = 1 \rightarrow 3 \rightarrow 5 \rightarrow 6 = t$ út növelő út 2 növekménnyel. Majd ezt után a $s = 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 6 = t$ is növelőút 1 növekménnyel. Így kapunk egy 5 értékű folyamat.

Hogyan láthatjuk be, hogy egy megengedett folyam maximális?

Azt mondjuk, hogy a gráf éleinek egy R halmaza ($R \subseteq E$) $s - t$ vágása a G gráfnak, ha az R -beli élek eltávolítása azt eredményezi, hogy nem lesz $s \rightsquigarrow t$ út.

Egy $s - t$ vágás értékén a vágási éleinek kapacitásainak összegét értjük.

Ha R egy $s - t$ vágás, akkor bármely 1 értékű folyam átmegy valamely R -beli élen.

Tehát a maximális folyam érték nem lehet nagyobb, mint a vágások minimális értéke.

Másképpen fogalmazva, van olyan $S \subseteq V$ és $T \subseteq V$ diszjunkt részhalmazok, hogy $V = S \cup T$, $s \in S$, $t \in T$, a vágás azon $(u, v) \in E$ éleket tartalmazza, amelyre $u \in S$ és $v \in T$.

Definiáljuk az (S, T) vágás kapacitását:

$$C(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

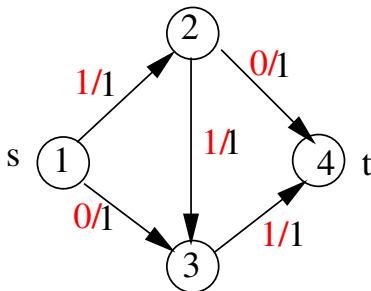
Definiáljuk az (S, T) vágáshoz tartozó folyam értékét:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

Látható hogy $|f| = f(\{s\}, V - \{s\})$.

Maximális folyam elvi algoritmus

1. lépés: Vegyünk egy tetszőleges $\pi = s \rightsquigarrow t$ utat és legyen k a π útba eső élek kapacitásának minimuma. Tehát π növelő út lesz k növekménnyel.
2. lépés: Csökkentsük a π úton lévő élek kapacitását k értékkel.
3. lépés: Ismételjük az 1-2 lépéseket amíg létezik növelőút



4. ábra. Egyetlen növelőút, ami nem optimális folyam

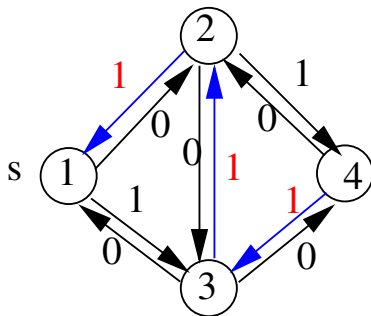
Definiáljuk f a $G_r = (V, E_r)$ **reziduális gráf** fogalmát az alábbiak szerint:

$$E_r = E + \{ (u, v) : (v, u) \in E \}$$

Kezdetben $c_r(u, v) = c(u, v)$ ha $(u, v) \in E$, egyébként

Egy k növekményű $\pi : s \rightsquigarrow t$ növelő út végrehajtása után

$$c_r(u, v) = \begin{cases} c_r(u, v) - k & \text{ha } (u, v) \in \pi \\ c_r(u, v) = c_r(u, v) - k & \text{ha } (v, u) \in \pi \\ c_r(u, v) = c_r(u, v) & \text{ha } (v, u) \notin \pi \end{cases}$$



5. ábra. A reziduális gráf az $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ 1-növelő út után.

```
1  while (van  $k > 0$  növekményű növelő út){  
2      a növelő úton módosítsuk a kapacitást;  
3      max_folyam += k;  
4  }
```

Ford-Fulkerson algoritmus tankönyvi megvalósítása

```
1 //Ford-Fulkerson algoritmus tankönyvi megvalósítása
2 #include <bits/stdc++.h>
3 #define INF 1000000000
4 using namespace std;
5 int n;
6 vector<vector<int>> capacity;
7 vector<vector<int>> adj;
```

```

1  int bfs(int s, int t, vector<int>& parent) {
2      fill(parent.begin(), parent.end(), -1);
3      parent[s] = -2;
4      queue<pair<int, int>> q;
5      q.push({s, INF});
6      while (!q.empty()) {
7          int cur = q.front().first;
8          int flow = q.front().second;
9          q.pop();
10         for (int next : adj[cur]) {
11             if (parent[next]==-1 && capacity[cur][next]){
12                 parent[next] = cur;
13                 int new_flow=min(flow, capacity[cur][next]);
14                 if (next == t)
15                     return new_flow;
16                 q.push({next, new_flow});
17             }
18         }
19     }
20     return 0;
21 }

```

```

22  int maxflow(int s, int t) {
23      int flow = 0;
24      vector<int> parent(n);
25      int new_flow;
26
27      while (new_flow = bfs(s, t, parent)) {
28          flow += new_flow;
29          int cur = t;
30          while (cur != s) {
31              int prev = parent[cur];
32              capacity[prev][cur] -= new_flow;
33              capacity[cur][prev] += new_flow;
34              cur = prev;
35          }
36      }
37
38      return flow;
39  }

```

```
22 void Readln(){}  
23  
24 int main(){  
25     int s, t;  
26     Readln();  
27     cout<<maxflow(s, t) <<endl;  
28 }
```

Ford-Fulkerson algoritmus: szomszédsági mátrix, mélységi bejárással

```
1 #include <bits/stdc++.h>
2 #define maxN 1001
3 #define maxKap 1000001
4
5 using namespace std;
6 typedef int Graf[maxN][maxN]; //súlyozott gráf
7 Graf G;
8 int Szin[maxN];
9 int n, t, s, aszin=0;
```

```

10 int NoveloUt(int u, int minkap){
11 // Global: G,t,aszin
12 //s→p úton minkap a minimum kapacitás
13     Szin[u]=aszin;
14     if(u==t) return minkap;
15     for(int v=1; v<=n; v++){
16         int cuv=G[u][v];
17         if(Szin[v]<aszin && cuv>0){
18             int vnov=NoveloUt(v, min(minkap,cuv));
19             if(vnov>0){
20                 G[v][u]+=vnov;
21                 G[u][v]-=vnov;
22                 return vnov;
23             }
24         }
25     }
26     return 0;
27 }

```



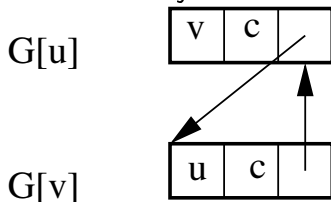
```

28  int main(){
29      int max_folyam=0;
30      int m,u,v,cuv;
31      cin>>n>>m>>s>>t;
32      for(int u=1;u<=n;u++){
33          Szin[u]=0;
34          for(int v=1;v<=n;v++)
35              G[u][v]=0;
36      }
37      for(int i=0;i<m;i++){
38          cin>>u>>v>>cuv;
39          G[u][v]=cuv;
40      }
41      for(;;){
42          aszin++;
43          int nov=NoveloUt(s, maxKap);
44          if(nov==0) break;
45          max_folyam+=nov;
46      }
47      cout<<max_folyam<<endl;
48  }

```

A szomszédsági mátrix ábrázolás pazarló, ha az élek száma jóval kisebb, mint n^2 .

A súlyozott gráfok standard szomszédsági listás ábrázolása azonban nem elégséges, mert $u \rightarrow v$ él alkalmazásakor a $v \rightarrow u$ élet is módosítani kell, tehát el kell tudni érni konstans időben. A megoldást a következő ábra mutatja.



6. ábra. $uvi = G[u].size()$, $vui = G[v].size()$ az (u, v) él bevételekor.

Ford-Fulkerson algoritmus: szomszédsági lista, mélységi bejárás

```
1 #include <bits/stdc++.h>
2 #define maxN 100001
3 #define maxKap 1000001
4 using namespace std;
5 struct El{
6     int v,c,vu;
7 };
8 typedef vector<El> Graf[maxN]; //súlyozott gráf
9 Graf G;
10 int Szin[maxN];
11 int n,t,s,aszin=0;
```

```

29 int NoveloUt(int u, int minkap){
30 // Global: G,t, aszin
31 // s→u úton minkap a minimum kapacitás
32     Szin[u]=aszin;
33     if(u==t) return minkap;
34     for(El &euv: G[u]){ //u→v élre
35         int cuv=euv.c;
36         int v=euv.v;
37         if(Szin[v]<aszin && cuv>0){
38             int vnov=NoveloUt(v, min(minkap, cuv));
39             if(vnov>0){
40                 G[v][euv.vu].c+=vnov;
41                 euv.c-=vnov;
42                 return vnov;
43             }
44         }
45     }
46     return 0;
47 }

```

```

48  int main() {
49      int max_folyam=0;
50      int m,u,v,cuv;
51      cin>>n>>m>>s>>t;
52      for(int i=0;i<m;i++){
53          cin>>u>>v>>cuv;
54          int uvi=G[u].size();
55          int vui=G[v].size();
56          G[u].push_back({v,cuv,vui});
57          G[v].push_back({u,0,uvi});
58      }
59      for(;;){
60          aszin++;
61          int nov=NoveloUt(s, maxKap);
62          if(nov==0) break;
63          max_folyam+=nov;
64      }
65      cout<<max_folyam<<endl;
66  }

```

Az algoritmus futási ideje legrosszabb esetben $O(|E| \cdot mf)$, ahol mf a maximális folyam értéke.

Tekintsük az algoritmus végrehajtása utáni **reziduális** gráfot. Ekkor biztosan nincs olyan $s \rightsquigarrow t$ út, ahol minden él pozitív értékű lenne. Legyen S az s -ből a reziduális gráfban nem 0 kapacitású éleken elérhető pontok halmaza, T pedig a többi.

Legyen $k=C(S,T)$ kapacitás az eredeti gráfban. Tudjuk a korábbiak alapján, hogy $|f| = f(S, T) \leq k$ (f az algoritmus által előállított folyam)

Tekintsük az eredeti G gráf összes olyan élet, amely keresztezi az S/T vágást valamelyik irányban. Két eset lehet.

2. eset

Tekintsük azokat az $u \rightarrow v$ éleket, ahol $u \in T$ és $v \in S$. Azt állítjuk, hogy az ilyen élek kapacitása 0. Tegyük fel az ellenezőjét. Ez azt jelenti, hogy van a fordított irányban pozitív kapacitású él S -ből T -be. Ez azt jelentené, hogy van S -beli pont, amelyből elérhető t . Tehát nincs él T -ből S -be.

Tehát azt kaptuk, hogy minden S -ből T -be menő (u, v) él esetén $f(u, v) = c(uv)$ és minden T -ből S -be vezető (u, v) élre $f(u, v) = 0$. Tehát a vágáshoz tartozó folyam megegyezik a $C(S, T)$ -vel. Mivel minden folyam értéke legfeljebb a vágások minimális kapacitása, ezért ez a vágás minimális kell legyen, és a folyam értéke megegyezik ezzel.

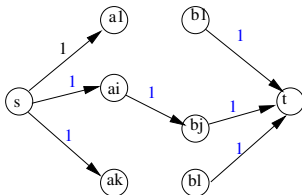
Maximális folyam, minimális vágás

A maximális folyam értéke megegyezik a vágások kapacitásának minimális értékével.

Maximális folyam algoritmus alkalmazásai

A probléma megoldható Ford-Fulkerson algoritmussal.

Legyen $G = (V, E)$ páros gráf, ahol $V = A + B$. Vegyünk fel két extra pontot, s -t és t -t és minden $a \in A$ pontra (s, a) , és minden $b \in B$ pontra (b, t) új éleket. Minden él kapacitása legyen 1.



8. ábra. Páros gráf folyammá alakítva

Ebben a módosított gráfban a maximális folyam értéke megegyezik a maximális párosítás értékével.

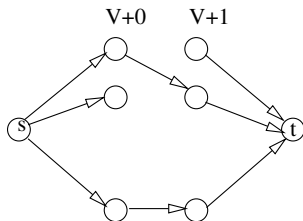
Ha a maximális párosításban szereplő minden (a_i, b_j) él esetén az $s \rightarrow a_i \rightarrow b_j \rightarrow t$ út növelő út lesz. Több növelő út nem lehet, mert akkor a párosítás nem lenne maximális.

Bemenet: $G(V, E)$ irányított, körmentes gráf.

Kimenet: A legkevesebb olyan pont-diszjunk utak száma, amelyek lefedik a V ponthalmazt, tehát minden pont pontosan egy úton van rajta. Lehet olyan út is, amely egyetlen pontot tartalmaz

Megoldás:

Tekintsük azt a $\overline{G} = (\overline{V}, \overline{E}, \overline{c})$ súlyozott gráfot, ahol



9. ábra

$$\overline{V} = \{ (v, 0), (v, 1) : v \in V \} \cup \{s, t\} \quad (1)$$

$$\overline{E} = \{ (u, 0) \rightarrow (v, 1) : (u, v) \in E \} \quad (2)$$

$$\cup \{ (s, (u, 0)) \} \cup \{ ((u, 1), t) \} \quad (3)$$

Minden él kapacitása legyen 1.

A minimális lefedő utak száma $|V| - m$, ahol m a maximális folyam értéke.

A lefedő utak előállíthatók úgy, hogy vesszük azt a gráfot, amelyben (u, v) akkor és csak akkor él, ha $((u, 0), (v, 1))$ szerepel a párosításban.

Adott az X és Y diszjunkt halmazok, továbbá az alábbi függvények.

$$c_x : X \rightarrow \mathbb{N}$$

$$c_y : Y \rightarrow \mathbb{N}$$

$$c_{xy} : X \times Y \rightarrow \mathbb{N}$$

Feladat: Adjunk meg egy legnagyobb elemszámú olyan $R \subseteq X \times Y$ multihalmazt, amelyre teljesül a következő három feltétel:

$$|\{ (a, y) \in R \}| \leq c_x(a)$$

$$|\{ (x, b) \in R \}| \leq c_y(b)$$

$$|\{ (a, b) \in R \}| \leq c_{xy}(a, b)$$

(Látható, hogy a probléma a maximális párosítás páros gráfban általánosítása.)

Megoldás: Visszavezetés maximális folyam megoldásra.

Alkossuk meg azt a $G(V, E, c)$ súlyozott gráfot, ahol

$$V = X + Y + \{s, t\}$$

$$E = \{(s, x) : x \in X\} \cup \{(y, t) : y \in Y\} \cup \{(x, y) : cxy$$

$$c(s, x) = cx(x)$$

$$c(y, t) = cy(y)$$

$$c(x, y) = cxy(x, y)$$

Számítsuk ki G maximális folyamát, legyen ez f .

Ekkor a keresett $R \subseteq X \times Y$ reláció azon x, y párokból áll, amelyek a maximális folyamban $s \rightarrow x \rightarrow y \rightarrow t$ utak.

Ezt a problémát hozzárendelési feladatnak is nevezik.

Két független út irányítatlan gráfban

Adott a $G = (V, E)$ irányítatlan gráf és az $A, B \in V$ pontja. Adjunk meg olyan $\pi_1 : A \rightsquigarrow B$ és $\pi_2 : A \rightsquigarrow B$ utakat, amelyeknek csak A és B a közös pontjuk. Jelezzük, ha nincs megoldás.

Megoldás

Tekintsük azt a $\overline{G} = (V, \overline{E}, A, B)$ maximális folyam problémát, ahol A a forrás, B a nyelő és $(u, v) \in \overline{E}$ akkor és csak akkor, ha $(u, v) \in E$ vagy $(v, u) \in E$. (Tehát G minden éle mindkét irányítással szerepel $\overline{E}$ -ben.

Legyen minden él kapacitása 1.

Állítás: akkor és csak akkor van két független $A \rightsquigarrow B$ út, ha a maximális folyam értéke legalább 2.

Tehát elegendő kétszer futtatni a Növelőút algoritmust.

Feladatok

1. feladat: Download Speed

<https://cses.fi/problemset/task/1694>

Egy számítógépes hálózat n csomópontot tartalmaz. m csomópontpár között van kiépítve egyirányú adatátvitelt biztosító közvetlen vonal. Minden közvetlen vonal meghatározott átviteli sebességgel tud adatcsomagokat továbbítani. Ha ez az érték c akkor az azt jelenti, hogy egy másodperc alatt c adatcsomagot tud továbbítani a másik csomópontba.

Ha egy adatcsomag megérkezik egy csomópontba, akkor időveszteség nélkül továbbítható bármely másik szomszédos csomópontba.

Készíts programot, amely kiszámítja, hogy az 1 csomópontból az n csomópontba legjobb esetben hány adatcsomag továbbítható egy másodperc alatt!

Bemenet

A standard bemenet első sorában a csomópontok n száma és a közvetlen vonalak m száma van. A következő m sor mindegyike három egész számot tartalmaz, a , b és c , ami egy közvetlen vonal adatai, azt jelenti, hogy az a csomópontból a b csomópontba egy másodperc alatt c adatcsomag továbbítható.

Kimenet

A standard kimenetre azt a legnagyobb k egész számot kell kiírni, amennyi adatcsomag továbbítható egy másodperc alatt az 1 csomópontból az n csomópontban!

Példa

Bemenet

```
4 5
1 2 3
2 4 2
1 3 4
3 4 5
4 1 3
```

Kimenet

6

Korlátok

Időlimit: 1.0 mp.

Memórialimit: 512 MB

2. feladat: Police Chase

<https://cses.fi/problemset/task/1695>

Adott a $G = (V = \{1, \dots, n\}, E)$ irányítatlan gráf.

Legkevesebb hány élet kell törölni a gráfból, hogy ne legyen út 1-ből n -be? Adjunk is meg egy megoldást!

Bemenet

A standard bemenet első sorában a pontok n és az élek m száma van. A további m sor egy-egy élet ad meg a két végpontjával.

Kimenet

A standard kimenetre legkevesebb törlendő élek k számát kell írni. A következő k sor egy-egy törlendő él két végpontját tartalmazza. Több megoldás esetén bármelyik megadható.

Példa

Bemenet

4 5

1 2

1 3

2 3

3 4

1 4

Kimenet

2

3 4

1 4

Korlátok

$$2 \leq n \leq 500$$

$$1 \leq m \leq 1000$$

$$1 \leq a, b \leq n$$

Időlimit 1.0 mp

Memória limit 512 MB

3. feladat: POTHOLE - Potholers

`https://www.spoj.com/problems/POTHOLE/`
Adott a $G = (V = \{1, \dots, n\}, E)$ irányítatlan gráf. Számítsuk ki, hogy legjobb esetben hány olyan 1-ből n -be vezető $p_1, \dots, p_k$ utat tudunk megadni, amelyeknek nincs közös éle. Minden útra teljesülnie kell, hogy minden útbeli $a \rightarrow b$ él esetén $a < b$.

Bemenet

A standard bemenet első sorában a tesztesetek t száma van. Minden teszteset első sorában a pontok n száma van. A további $n - 1$ sor adja meg a gráf éleit. Az $i + 1$ -edik sor tartalmazza azokat a j pontokat, amelyekre $(i, j) \in E$. Minden sorban az első szám az (i, j) élek száma.

Kimenet

A standard kimenetre pontosan t sort kell írni, az i -edik sorba az i -edik teszteset megoldási értékét.

Példa

Bemenet	Kimenet
---------	---------

1	3
---	---

12	
----	--

4 3 4 2 5	
-----------	--

1 8	
-----	--

2 9 7	
-------	--

2 6 11	
--------	--

1 8	
-----	--

2 9 10	
--------	--

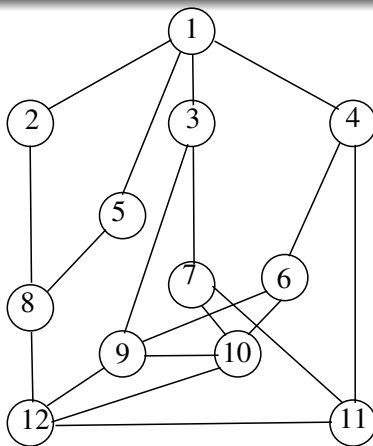
2 10 11	
---------	--

1 12	
------	--

2 10 12	
---------	--

1 12	
------	--

1 12	
------	--



10. ábra

Korlátok

$$2 \leq n \leq 500$$

$$1 \leq m \leq 1000$$

$$1 \leq a, b \leq n$$

Időlimit 1.0 mp

Memória limit 512 MB